1. Fixed Window Limiter (`FixedWindowLimiter`)

- **The Approach:** Divides time into static, non-overlapping windows. It maps incoming requests to a discrete time slot and increments an atomic request counter. When the window boundary is crossed, the counter resets. [1, 2]
- **Time Complexity:**
  - **Best/Average Case: $\mathcal{O}(1)$** — Under low contention, a single Compare-And-Set (CAS) loop iteration successfully increments the counter.
  - **Worst Case: $\mathcal{O}(\text{threads})$** — Under heavy thread contention, threads will spin in the `while(true)` loop failing the CAS operation until they successfully update the state.
- **Space Complexity: $\mathcal{O}(1)$ (Constant Space)**
  - The memory footprint is strictly fixed. The `WindowState` record only holds two primitive values (`long start` and `int count`), requiring less than **32 bytes** of heap allocation per client, regardless of the maximum request volume or window size. [1, 2]

---

2. Sliding Window Limiter (`SlidingWindowLimiter`)

- **The Approach:** Tracks a rolling window log by storing a timestamp for every individual successful request. It dynamically filters out old timestamps older than `now - winNs` upon every request entry.
- **Time Complexity:**
  - **Best/Average Case: $\mathcal{O}(N)$** — Where N is the number of active requests currently stored in the window (bounded by `maxReq`). For every incoming request, the code executes a linear loop to filter out expired timestamps and a second loop to build the replacement array.
  - **Worst Case: $\mathcal{O}(N \times \text{threads})$** — Under extreme thread contention, multiple threads pass the limit check, copy the array, and race to CAS. The losing threads discard their copied array, loop back, and re-scan the timestamp array from scratch. [1, 2]
- **Space Complexity: $\mathcal{O}(N)$ (Linear Space)**
  - Where N is the number of active requests (at most `maxReq`). The immutable `SlidingState` stores an array containing exactly the timestamps of unexpired requests. Memory usage scales directly with your maximum request capacity threshold.

---

3. Token Bucket Limiter (`TokenBucketLimiter`)

- **The Approach:** Utilizes a lazy-refill accumulator model. Instead of maintaining a background daemon thread to add tokens, it calculates fractional token generation dynamically whenever a request hits the system using fixed-point integer scaling.
- **Time Complexity:**
  - **Best/Average Case: $\mathcal{O}(1)$** — Performs simple mathematical primitive operations (multiplication and division) and applies them via a standard single-step atomic reference swap.
  - **Worst Case: $\mathcal{O}(\text{threads})$** — Under high concurrent request volumes, losing threads spin in the loop. However, your **clock forward-alignment optimization** (`Math.max(now, current.lastRefillTime)`) keeps these retries incredibly lightweight by bypassing expensive system clock hardware queries if another thread has already updated the base timeline. [1]
- **Space Complexity: $\mathcal{O}(1)$ (Constant Space)**
  - The memory structure remains fixed forever. The `BucketState` record stores exactly three primitive elements (`long tokensScaled`, `long lastRefillTime`, and `long remainderNs`), keeping heap allocations bounded to minimal bytes per client. [1, 2]

Complexity Comparison Matrix

Rate Limiter Variant [1, 2, 3, 4]	Time Complexity (No Contention)	Time Complexity (High Contention)	Space Complexity	GC Allocation Profile
Fixed Window	$\mathcal{O}(1)$	$\mathcal{O}(\text{threads})$	$\mathcal{O}(1)$	**Zero Churn** (Reuses/Creates small records only on success)
Sliding Window	$\mathcal{O}(N)$	$\mathcal{O}(N \times \text{threads})$	$\mathcal{O}(N)$	**High Churn** (Allocates new `long[]` arrays on every single loop retry)

Token Bucket	$\mathcal{O}(1)$	$\mathcal{O}(\text{threads})$	$\mathcal{O}(1)$	**Zero Churn** (Reuses/Creates small records only on success)

**Note: N represents the maximum number of requests allowed within the window (`maxReq`).*